

ERI - TP FINAL - PRESENTACIÓN

Santiago Rocca - Maximiliano Fisz

July 31, 2025

Objetivo

Clasificaremos imagenes segun que tipo de vehiculo se encuentra como "sujeto principal" en ella.



Motivaciones

- Probar otras herramientas de extraccion de descriptores y clusterizacion para compararlas con el categorizador de ambientes.

Motivaciones

- Probar otras herramientas de extracción de descriptores y clusterización para compararlas con el categorizador de ambientes.
- Aprender a refinar el proceso para conseguir resultados "utilizables" en proyectos reales.

Deteccion de features y extraccion de descriptores

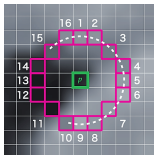
ORB (Oriented FAST and Rotated BRIEF).

- Similar a SIFT en performance.
- Invariante a la rotacion.
- Muy capaz con bajos recursos de computación.
- Codigo abierto.

ORB

Keypoints

- Usar FAST para encontrar keypoints (Diferencias en un circulo de Bresenham).
- Quedarnos con los keypoints que respondan más al detector de esquinas de Harris.
- Asignarle una orientacion a cada keypoint segun la dirección de un vector desde el centro de este al centro de intensidad del area vecina.



Descriptores

- Rotar el area que usara BRIEF con la orientacion encontrada antes.
- Usar BRIEF para calcular descriptores.
 - Usar los keypoints ya elegidos.
 - Tomar N pares de puntos (p, q) en un vecindario de estos.
 - Para cada par, su valor en el descriptor es:

$$\text{Bit}_i = \begin{cases} 1, & \text{si } I(p) < I(q) \\ 0, & \text{si no} \end{cases}$$

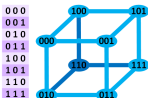
- Concatenar y generar el vector binario de tamaño N.

Clusterizacion

Tambien utilizaremos Bag Of Words pero usaremos otra forma de crear el vocabulario visual: K-Medoids.

- K-Medoids solo puede tomar datos como centroides en vez de cualquier punto.
- Permite calcular las distancias con metricas diferentes a la euclidiana (¡tenemos descriptores binarios!).
- Posibles Implementaciones:
 - PAM (Partitioning Around Medoids): Algoritmo original, muy costoso en terminos de memoria.
 - CLARA (Clustering Large Applications): Aplica el algoritmo original pero a subconjuntos al azar de los datos.

3 Dimensiones



Modelos

Buscamos entonces el mejor modelo para nuestro problema de clasificacion entre los posibles candidatos:

- ORB + PAM
- ORB + CLARA
- ORB + K-Means
- SIFT + K-Means

Modelos

Buscamos entonces el mejor modelo para nuestro problema de clasificacion entre los posibles candidatos:

- ~~ORB + PAM~~ (insuficientes recursos: 500 keyp. requiere $\approx$ 1TB de RAM)
- ORB + CLARA
- ORB + K-Means
- SIFT + K-Means

Metodología

Tenemos:

- Clases: auto, pickup, motocicleta o bicicleta.
- Datos: 1200 imagenes total, convertidas a escalas de grises y resizeadas a 512x512p.
- Dividimos el dataset en 80% para entrenar y el resto para validar.
- Buscamos los mejores parametros para cada modelo haciendo cross-validation entre 3 subdivisiones del conjunto de entrenamiento.
- Para cada prueba entrenamos una SVM y la probamos con el 1/3 designado para validar.
- Se seleccionan los mejores resultados para ese modelo y se entrena un modelo final.
- Comparamos los resultados finales de cada modelo.

ORB + CLARA - Preparativos

En este modelo, debemos indicarle a ORB la cantidad de keypoints que debe detectar en la imagen, además, elegir la cantidad de palabras visuales del corpus.

Probamos con los keypoints:

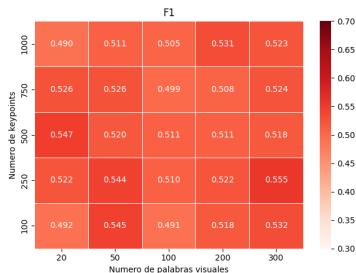
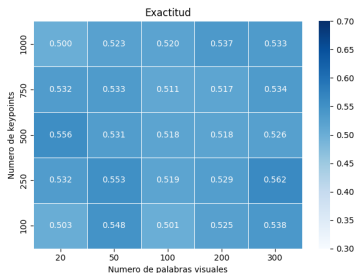
- 100, 250, 500, 750 y 1000

y palabras visuales:

- 20, 50, 100, 200 y 300

ORB + CLARA - Selección del modelo final

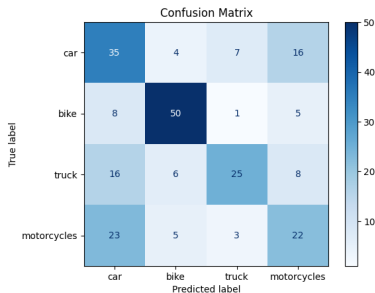
El cross validation nos dio los siguientes resultados:



Priorizando la exactitud, nos quedamos con 250 keypoints y 300 palabras visuales.

ORB + CLARA - Ejecución del modelo final

El resultado final evaluando contra el conjunto de validación:



Metrica	Exactitud	F1 promedio
Value	0.5641	0.5585

ORB + K-Means - Preparativos

En este modelo tambien, debemos indicarle a ORB la cantidad de keypoints que debe detectar en la imagen, ademas, elegir la cantidad de palabras visuales del corpus.

Probamos con los keypoints:

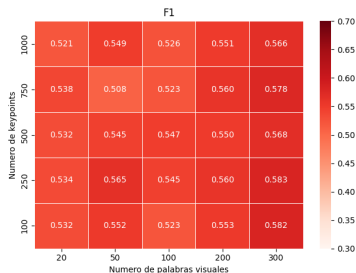
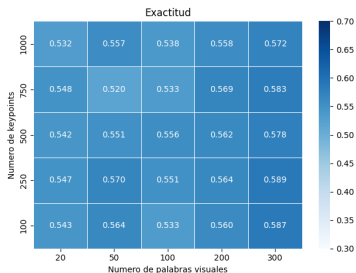
- 100, 250, 500, 750 y 1000

y palabras visuales:

- 20, 50, 100, 200 y 300

ORB + K-Means - Selección del modelo final

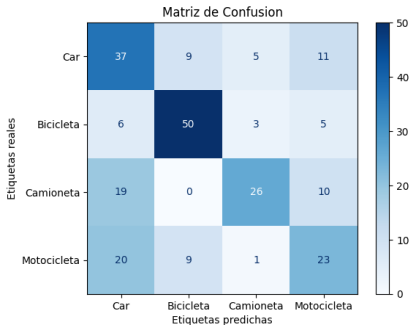
El cross validation nos dio los siguientes resultados:



Priorizando la exactitud, nos quedamos con 250 keypoints y 300 palabras visuales. ¡La misma combinación que para el modelo anterior!

ORB + K-Means - Ejecución del modelo final

El resultado final evaluando contra el conjunto de validación:



Metrica	Exactitud	F1 promedio
Value	0.6453	0.6372

SIFT + K-Means - Preparativos

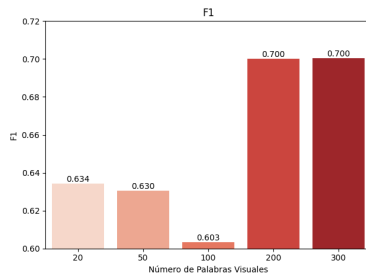
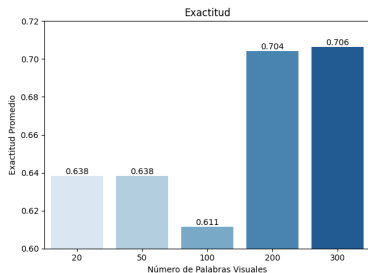
En este modelo, SIFT detecta todos los keypoints que puede segun su criterio. Decidimos usarlos todos. Solo debemos elegir la cantidad de palabras visuales del corpus.

Probamos con las palabras visuales:

- 20, 50, 100, 200 y 300

SIFT + K-Means - Selección del modelo final

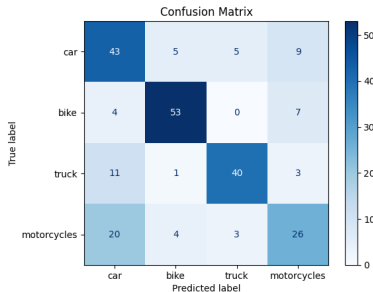
El cross validation nos dio los siguientes resultados:



Priorizando la exactitud, nos quedamos con 300 palabras visuales.
¡La misma combinación que para los modelos anteriores!

SIFT + K-Means - Ejecución del modelo final

El resultado final evaluando contra el conjunto de validación:



Metrica	Exactitud	F1 promedio
Value	0.6923	0.5585

Resumen de resultados

Model	Exactitud	F1 promedio
ORB + CLARA	0.5641	0.5585
ORB + K-Means	0.6453	0.6372
SIFT + K-Means	0.6923	0.6896

Obtenemos en general mejores resultados que en nuestro categorizador de ambientes (0.5 de exactitud) con todas las combinaciones.

Particularmente, el modelo que usa las mismas tecnicas tiene mejor precisión, teniendo menos de la mitad de las imagenes!
Consideramos (y sospechamos con evidencia) que podemos mejorar mucho más los resultados.

Posibles mejoras

Detectamos algunos posibles problemas y sus posibles mejoras/soluciones:

- Falta de información: nuestro dataset actual es muy pequeño, pruebas breves duplicando o triplicando el tamaño mejoran la clasificación notablemente.
- Confusion notable entre autos y motocicletas en todos los modelos: ya que la confusion solo sucede entre estas dos clases, podriamos primero clasificar segun numero de ejes y luego con mas precisión entre diferentes subclases.
- Curar el dataset

